

Assignment 2

- Assignment 2
 - Attribution
 - Completion Steps
 - * Create the `requirements.txt`
 - Create and activate a virtual environment
 - Install the required libraries in the virtual environment
 - Create the `requirements.txt` using the dependencies installed in the virtual environment
 - * Create the Service Application (`app.py`)
 - * Create the `Dockerfile` Image Blueprint
 - * Build An Image Tagged `dist-node:v1`
 - * Run Two Distinct Node Instances
 - * Verification
 - Discussion
 - * Prompt
 - * Answer

Summary: Create a containerized micro-service component by building and deploying a base image.

Attribution

Author

James Daniel Johnson

CWID

20183229

Course

CSIS 604 - Distributed Systems

Assignment

2 - Containerizing a Distributed Micro-Node Service

Completion Steps

Create the `requirements.txt`

These steps don't represent the *only* way to create a `requirements.txt` for use with a `Dockerfile`; however, they do represent a fairly standard workflow for iterative creation and maintenance of dependency requirements since it automatically updates the `requirements.txt` by running `pip freeze` and directing the output into a `requirements.txt` file. This allows automated CI/CD pipelines (such as GitHub workflows or Jenkins) to replicate and deploy images deterministically by following these steps.

Create and activate a virtual environment

```
python -m venv .venv
source .venv/bin/activate
```

Install the required libraries in the virtual environment This keeps the required libraries and versions coupled to the project or application as opposed to the global python environment. In this case the only required library is `Flask`.

```

$ python -m pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (23 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (23 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.5.0 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

```

Create the **requirements.txt** using the dependencies installed in the virtual environment This completes the cycle. Whenever a new library is installed or updated, this can be used to maintain consistency amongst requirements, ensuring portability of the image across versions.

```

$ python -m pip freeze > requirements.txt
$ cat requirements.txt
blinker==1.9.0
click==8.5.0
Flask==3.1.3
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
Werkzeug==3.1.8

```

Create the Service Application (**app.py**)

```

# Author: James Daniel Johnson
# CWID: 20183229
# Course: CSIS 604 - Distributed Systems
# Assignment: 2.1 Python REST service

import os
import socket

from flask import Flask, Response, jsonify

app = Flask(__name__)

# Use the environment variables to set node identity
NODE_ID = os.environ.get("NODE_ID", "default-node")

```

```

PORT = int(os.environ.get("PORT", "5000"))

@app.route("/", methods=["GET"])
def index() -> tuple[Response, int]:
    return jsonify(
        {
            "status": "online", # aka "up"
            "node_id": NODE_ID,
            "hostname": socket.gethostname(),
        }
    ), 200

@app.route("/health", methods=["GET"])
def health() -> tuple[Response, int]:
    return jsonify({"status": "healthy"}), 200

if __name__ == ("__main__"):
    app.run(host="0.0.0.0", port=PORT) # Listens on all available addresses

```

Create the Dockerfile Image Blueprint

```

# Use a lightweight official python runtime
FROM python:3.13-slim

# Set the working directory inside the container
WORKDIR /app

# Copy requirements.txt to the working directory
COPY requirements.txt .
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy app.py into the container
COPY app.py .

# Expose port 5000
EXPOSE 5000

# Set default NODE_ID
ENV NODE_ID=node-01

# Set default PORT
ENV PORT=5000

# Set the entrypoint of the application
CMD ["python", "./app.py"]

```

Now we should have all of the required files to build our image.

```

$ tree .
.
├─ app.py

```

```
├─ Dockerfile
├─ README.md
└─ requirements.txt
```

Build An Image Tagged `dist-node:v1`

```
$ sudo docker build -t dist-node:v1 .
```

Place your finger on the fingerprint reader

```
[+] Building 8.8s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 542B
=> [internal] load metadata for docker.io/library/python:3.13-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.13-slim@sha256:7c61056e61ac89e852de05f3dc6fa51a6dd2181797bceed46aa725dd
=> => resolve docker.io/library/python:3.13-slim@sha256:7c61056e61ac89e852de05f3dc6fa51a6dd2181797bceed46aa725dd
=> => sha256:4a43a40b039e72c4176e2fcb0c08f42be7ff4e6a07722417801c9147b67bfb2 250B / 250B
=> => sha256:264ba3d8ae19c964ad7ebecdd4379a348cc08c7898b50d08d7884127a78fdb23 11.91MB / 11.91MB
=> => sha256:3d9fb74714202c0d9ba7af2ca3e226fec5647f73cefc725b7ff37338bc1b59ec 1.29MB / 1.29MB
=> => extracting sha256:3d9fb74714202c0d9ba7af2ca3e226fec5647f73cefc725b7ff37338bc1b59ec
=> => extracting sha256:264ba3d8ae19c964ad7ebecdd4379a348cc08c7898b50d08d7884127a78fdb23
=> => extracting sha256:4a43a40b039e72c4176e2fcb0c08f42be7ff4e6a07722417801c9147b67bfb2
=> [internal] load build context
=> => transferring context: 1.04kB
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:9f5a7978da652c2c34494fc6a02ec011e8ddacf2af5881ca851a087ee07d4ea6
=> => exporting config sha256:33c08e78ba442cd34a649bccfdce42d020e14df7c89604e7b3b9f93f05b66d95
=> => exporting attestation manifest sha256:aebb1a17c4136841277c0188dac40209cd4143e83c51e69700b09e6eb9b3ad6e
=> => exporting manifest list sha256:34f4a903345afa8813779b18660f58afccccc9f0352c76209d36bcb63341fe4a4
=> => naming to docker.io/library/dist-node:v1
=> => unpacking to docker.io/library/dist-node:v1
```

Run Two Distinct Node Instances

```
$ sudo docker run -d --name node1 -p 8081:5000 -e NODE_ID="worker-node-alpha" dist-node:v1
```

Place your finger on the fingerprint reader

```
9da97eca5de2d31268f7a9ac1cd14f556d7f3ad77c8e31da41001111225af500
```

```
$ sudo docker run -d --name node2 -p 8082:5000 -e NODE_ID="worker-node-beta" dist-node:v1
```

```
eee3364b5a4cf68922b066c9bde6d01205106dc4c1265afb6eb964a57b426d29
```

Verification

```
$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
9da97eca5de2	dist-node:v1	"python ./app.py"	2 hours ago	Up 5 seconds	0.0.0.0:8081->5000/tcp, [::]:8081->5000/tcp
eee3364b5a4c	dist-node:v1	"python ./app.py"	2 hours ago	Up 5 seconds	0.0.0.0:8082->5000/tcp, [::]:8082->5000/tcp

```
$ curl http://localhost:8081/{1,2}/{health}
{"hostname": "9da97eca5de2", "node_id": "worker-node-alpha", "status": "online"}
{"status": "healthy"}
{"hostname": "eee3364b5a4c", "node_id": "worker-node-beta", "status": "online"}
```

```

{"status":"healthy"}
$ for i in {1,2} ; do
for>   for suffix in {,health} ; do
for for>   curl http://localhost:8080$i/$suffix | jq '.'
for for>   done
for> done
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    76 100    76    0      0  35967      0         0         0
{
  "hostname": "9da97eca5de2",
  "node_id": "worker-node-alpha",
  "status": "online"
}
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    21 100    21    0      0  10926      0         0         0
{
  "status": "healthy"
}
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    75 100    75    0      0  33860      0         0         0
{
  "hostname": "eee3364b5a4c",
  "node_id": "worker-node-beta",
  "status": "online"
}
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    21 100    21    0      0  10943      0         0         0
{
  "status": "healthy"
}

```

Discussion

Prompt

In 2–3 sentences, explain why we set `host="0.0.0.0"` in `app.py` instead of `host="127.0.0.1"` when binding a server inside a Docker container.

Answer

The *semantic meaning* of setting a host to the *wildcard address* (aka `"0.0.0.0"` in IPv4 or `[::]` in IPv6) is “this host listens on all available addresses assigned to this machine”. Since this application is containerized, its loopback address (resolved from `localhost`, `127.0.0.1` using IPv4, `::1` using IPv6) only point to the process running *inside the container*. Hence, if we want to be able to reach it from the host machine (or an outside network via port forwarding), we need it to listen to all of its available addresses, so we set it to the wildcard address.